

SINTAXIS DEL PSEUDOLENGUAJE DE DESCRIPCIÓN DE HARDWARE

Ángel Serrano Sánchez de León

Para la descripción de los componentes de los ejercicios se puede usar el siguiente pseudolenguaje de descripción de hardware:

Tipos de datos:

```
bit a // Declaración
a = 1 // Asignación de valor (después de haberlo declarado)
bit b = 0 // Declaración y asignación de valor
bit c[2] // Declaración de una señal de 2 bits (array de bits)
c = 01 // c[1] vale 0, c[0] vale 1 (el orden de los bits es little-endian)
bit c[2] = 01 // Declaración y asignación de valor de una señal de 2 bits
```

Concatenación de bits (los bits se van colocando de más significativo (izquierda) a menos significativo (derecha) según van apareciendo en la expresión:

```
c = a || b // a y b son de 1 bit, c tiene 2 bits (c[1] = a, c[0] = b)
// Con los valores del ejemplo anterior (a = 1, b = 0), ahora c vale 10
```

Puertas lógicas: NOT a, a AND b, a OR b, a XOR b, a NAND b, a NOR b, a XNOR b

Definición de un bloque combinacional como si fuera una función:

```
(tipo salida1, tipo salida2, etc.) nombreDelBloque(tipo entrada1, tipo entrada2, etc.)
{
    // tipo puede ser bit o un array de bits, es decir, bit[]
    // Antes del return se calculan las salidas en función de las entradas
    return (salida1, salida2, etc.)
}
```

Sumador completo de 1 bit:

```
(bit Cout, bit S) sumadorCompleto_1_bit(bit a, bit b, bit cin)
{
    bit Cout = (cin AND b) OR (cin AND a) OR (a AND b)
    bit S = (a XOR b) XOR cin
    return (Cout, S)
}
```

Multiplexor de 2 a 1:

```
(bit result) mux2a1(bit entradas[2], bit control, bit enable)
{
    bit result = 0
    if (enable == 1) // La comprobación de igualdad es ==, la asignación es =
    {
        switch (control) // Se activa un caso en función del valor de control
        {
            case 0:
                result = entradas[0]
                break // Para saltarnos el resto de casos
            default:
                result = entradas[1]
        }
    }
    return (result)
}
```

Multiplexor de 4 a 1:

```
(bit result) mux4a1(bit entradas[4], bit control[2], bit enable)
{
    bit result = 0
    if (enable == 1)
    {
        switch (control)
        {
            case 00:
                result = entradas[0]
                break
            case 01:
                result = entradas[1]
                break
            case 10:
                result = entradas[2]
                break
            default:
                result = entradas[3]
        }
    }
    return (result)
}
```

Multiplexor de 8 a 1:

```
(bit result) mux8a1(bit entradas[8], bit control[3], bit enable)
{
    bit result = 0
    if (enable == 1)
    {
        switch (control)
        {
            case 000:
                result = entradas[0]
                break
            case 001:
                result = entradas[1]
                break
            case 010:
                result = entradas[2]
                break
            case 011:
                result = entradas[3]
                break
            case 100:
                result = entradas[4]
                break
            case 101:
                result = entradas[5]
                break
            case 110:
                result = entradas[6]
                break
            default:
                result = entradas[7]
        }
    }
    return (result)
}
```